

BioSims

Masterpiece voorstel Jos van Peperstraten (versie 3-6-2026)

Inhoud

Idee	1
Basisflow van de simulatie(s)	1
1 SimYeast / SimPolute	2
2. SimPlague	3
Mogelijke SimPlague scenario's	3
Prototype bouwen SimYeast/SimPolute	4
Basisvoorwaarden	4
Suggesties van co-pilot naar aanleiding van de data die ik wil hebben	5
Tabel met concrete startwaarden	6
Eenvoudig simulatiemodel in pseudocode	7
Visuele simulatie in Pygame	11
Eerste werkende Pygame-code	12
Klasse YeastCellSprite	15
Voorgestelde projectstructuur	16
Code opsplitsen over main.py, yeast.py en simulation.py	17
Pygame-code verbeteren met echte beweging	20
Klein grafiekvenster in Pygame	20

Idee

Ik wil een of enkele simulaties laten uitvoeren van biologische fenomenen, bij voorkeur op een visueel aantrekkelijke manier. Doel: leerlingen op een interactieve manier met het biologisch fenomeen laten experimenteren en kennismaken met modelleren.

Basisflow van de simulatie(s)

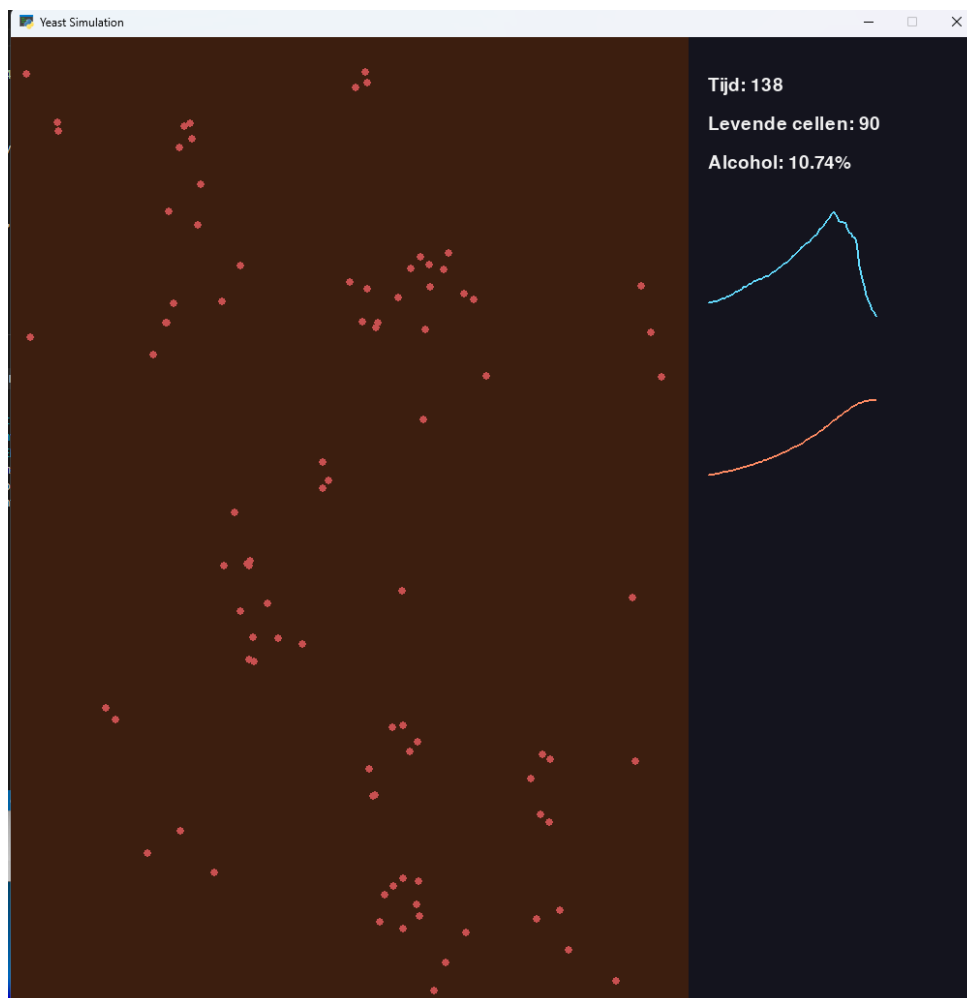
- Instellen van parameters in gui
- Waar mogelijk In py game simuleren (eventueel met live grafiek erbij)
- Data uitvoeren naar bestand (csv of opslaan in database)
- Plotten in meer gedetailleerde grafiek, J-curve? >> eventueel uitbesteden aan excel?

1 SimYeast / SimPolute

Gist in druivensap (uitproberen van de benodigde technieken, parameters hardcoded)

Onbeperkt voeding, gist vervuult omgeving met alcohol, gistcellen delen zich, kans op overlijden neemt toe bij toenemend alcoholpercentage.

Prototype gebouwd o.b.v. pygame. Om wat sneller te kunnen zien of dit werkbaar en haalbaar is heb ik daarbij ook de hulp ingeroepen van co-pilot (zie hieronder). Het resultaat is een werkend geheel, maar het is waarschijnlijk niet heel aantrekkelijk voor de gemiddelde leerling en de grote hoeveelheid objecten, die exponentieel groeit, lijkt op bepaalde momenten voor vertraging in de weergave te zorgen (misschien rond het moment van delen, dat lijkt nog wat pieksgewijs te verlopen). Het instellen van de parameters in een GUI en de export naar het databestand ontbreekt nog. Ook het gedetailleerd plotten van de grafiek ontbreekt nog.



Screenshot van prototype SimYeast.

2. SimPlague

Mijn oorspronkelijke idee en het einddoel.

Een visuele simulatie van een plaag (zo aantrekkelijk mogelijk vormgegeven), die de gegevens plot en hopelijk de J-curve laat zien met stabilisatie rond een nieuw evenwicht.

Mogelijke SimPlague scenario's

Scenario 1:

Grote hoeveelheid voedsel in gebied geen concurrentie, exoot introduceren, voortplanten tot draagkracht van het gebied. Geen /wel geslachtelijke voortplanting.

Zou kunnen leiden tot een j-curve met plotselinge sterfte als het aantal individuen van de exoot te groot wordt of tot een j-curve die overgaat in een evenwichtssituatie of massale sterfte

Scenario 2:

Grote hoeveelheid voedsel in gebied, voedselconcurrent plant trager voort, exoot introduceren, voortplanten tot draagkracht van het gebied,

Leidt mogelijk tot verdringen van de voedselconcurrent (afhankelijk van andere factoren zoals hoeveelheid benodigd voedsel, worpgrootte bij inheemse soort en de exoot, groei- en voortplantingssnelheid van het voedsel enz.), gevolgd door een nieuw evenwicht of massale sterfte.

Scenario 3

Grote hoeveelheid voedsel in gebied, voedselconcurrent plant trager voort, exoot introduceren, voortplanten tot draagkracht van het gebied, verdringen van de voedselconcurrent. Predator erbij.

Leidt hopelijk tot mooie grafieken waarbij de exoot, de voedselconcurrent en de predator uit fase lopen.

Prototype bouwen SimYeast/SimPolute

Basisvoorwaarden

Gistcel in druivensap

- Max levensverwachting (vast) (*enkele dagen tot weken*)
- stamina (variabel)
- Voortplantingssnelheid (ongeslachtelijk)
- Replicatieve levensduur (*20-30 delingen*)
- Produceert alcohol
- Stamina/energy wordt kleiner bij hoger percentage alcohol
- Stamina zorgt ook voor meer /minder productie alcohol
- Overlevingskans neemt af bij hogere concentratie alcohol
- Massale sterfte bij 12%

Basis voor de gistcel-objecten:

Class Yeastcel

__INIT__

lifeExpectancy

stamina

reproductionInterval

chanceOfSurvival (factoren: lifeExpectancy, stamina, alcoholPercentage,)

lethalAlcoholPercentage

alcoholProduction

Suggesties van co-pilot naar aanleiding van de data die ik wil hebben.

“Voor een simulatie met gist in druivensap is het vaak handiger om te werken met:

- een **maximale leeftijd** in tijdseenheden, bijvoorbeeld **3–7 dagen**
- én een **toenemende sterftekans** bij stijgend alcoholpercentage
- plus eventueel een limiet op het aantal delingen per cel, bijvoorbeeld **25 delingen**

Praktische modelkeuze

- **basislevensduur: 5 dagen**
- **max aantal delingen: 25**
- **extra sterftekans vanaf 8–10% alcohol**
- **massale sterfte rond 12–15% alcohol**
(afhankelijk van hoe sterke gist je wilt simuleren)

Omdat je al noemt:

- “massale sterfte bij 12%”
- “max levensverwachting (vast) en stamina (variabel)”

zou een goede eerste aanname zijn:

Levensverwachting gistcel: 3–7 dagen of maximaal 20–30 delingen, met sterk afnemende overlevingskans bij stijgend alcoholpercentage.”

Tabel met concrete startwaarden

Parameter	Startwaarde	Toelichting voor simulatie
Maximale levensduur	5 dagen	Praktische middenwaarde binnen een bereik van 3–7 dagen.
Replicatieve levensduur	25 delingen	Past binnen het vaak gebruikte bereik van circa 20–30 delingen.
Delingsinterval	90 minuten	Snelle deling onder gunstige omstandigheden; kan later afhankelijk gemaakt worden van stamina en alcohol.
Beginnende extra sterftekans	vanaf 8% alcohol	Vanaf dit punt neemt de overlevingskans merkbaar af.
Sterke rem op groei	vanaf 10% alcohol	Deling en alcoholproductie kunnen hier duidelijk trager worden.
Massale sterfte	12% alcohol	Goed passend bij je huidige uitgangspunt in het document.
Alcoholproductie per tijdseenheid	0,05–0,10% per uur per actieve populatie-eenheid	Modelwaarde; handig als schaalbare simulatieparameter in plaats van exacte biologische maat.
Stamina	100 start, daalt met alcohol	Bijvoorbeeld lineair of exponentieel laten afnemen naarmate alcohol stijgt.
Overlevingskans per stap	99% laag alcohol, daarna dalend	Bij lage alcohol bijna stabiel; daarna per stap verlagen op basis van alcoholpercentage en leeftijd.

Dit zijn **werkbare startwaarden** voor een eerste versie van de simulatie; daarna kunnen ze worden gefinetuned op basis van het gedrag van de grafiek en de populatiedynamiek.

Eenvoudig simulatiemodel in pseudocode

Onderstaand model werkt in vaste tijdstappen en volgt per stap hoeveel gistcellen leven, hoeveel alcohol is gevormd en hoeveel cellen zich nog kunnen delen.

START

```
zet tijd = 0
zet tijdStap = 1 uur
zet eindTijd = 200 uur
zet alcoholPercentage = 0
maak lijst populatie met 100 gistcellen
```

```
voor elke gistcel:
    leeftijd = 0
    aantalDelingen = 0
    maxLeeftijd = 120 uur
    maxDelingen = 25
    delingInterval = 1,5 uur
```

zolang tijd < eindTijd en populatie niet leeg:

```
    zet nieuweCellen = lege lijst
    zet levendeCellen = lege lijst
```

voor elke gistcel in populatie:

```
        verhoog leeftijd met tijdStap
```

```
        bereken overlevingskans
```

```
        als alcoholPercentage < 8 dan overlevingskans = 0,99
```

```
        als alcoholPercentage tussen 8 en 10 dan overlevingskans = 0,95
```

```
        als alcoholPercentage tussen 10 en 12 dan overlevingskans = 0,85
```

```
        als alcoholPercentage >= 12 dan overlevingskans = 0,20
```

```
        verlaag overlevingskans extra als leeftijd dicht bij maxLeeftijd
```

komt

```
        trek een willekeurig getal tussen 0 en 1
```

```
        als willekeurig getal < overlevingskans en leeftijd < maxLeeftijd:
```

```
            voeg gistcel toe aan levendeCellen
```

```
            als aantalDelingen < maxDelingen EN tijd modulo delingInterval
            = 0 EN alcoholPercentage < 10:
```

```
                maak nieuwe gistcel
```

```
                zet eigenschappen van nieuwe gistcel op beginwaarden
```

```
                verhoog aantalDelingen van moedercel met 1
```

```
        voeg nieuwe gistcel toe aan nieuweCellen

vervang populatie door levendeCellen + nieuweCellen

bereken alcoholToename = aantal levende cellen × 0,0005 per uur
als alcoholPercentage > 10:
    maak alcoholToename kleiner, bijvoorbeeld × 0,5

verhoog alcoholPercentage met alcoholToename

sla op voor grafiek:
    tijd
    aantal cellen
    alcoholPercentage

verhoog tijd met tijdStap

EINDE

plot aantal cellen tegen tijd
plot alcoholPercentage tegen tijd
```

Later is dit uit te breiden met stamina, een variabele delingssnelheid, beperkte suikervoorraad of verschillen tussen sterkere en zwakkere gistcellen.

Python-schets van de klasse YeastCell

Hieronder staat een eenvoudige eerste versie van de klasse

```
class YeastCell:
    def __init__(self, max_age=120, max_divisions=25, reproduction_interval=1.5,
alcohol_production=0.0005, lethal_alcohol=12):
        self.age = 0
        self.divisions = 0
        self.max_age = max_age
        self.max_divisions = max_divisions
        self.reproduction_interval = reproduction_interval
        self.alcohol_production = alcohol_production
        self.lethal_alcohol = lethal_alcohol
        self.stamina = 100

    def update_stamina(self, alcohol_percentage):
        if alcohol_percentage < 8:
            self.stamina = 100
        elif alcohol_percentage < 10:
            self.stamina = 80
        elif alcohol_percentage < 12:
            self.stamina = 50
        else:
            self.stamina = 10

    def chance_of_survival(self, alcohol_percentage):
        self.update_stamina(alcohol_percentage)
        base_chance = 0.99

        if alcohol_percentage >= self.lethal_alcohol:
            base_chance = 0.20
        elif alcohol_percentage >= 10:
            base_chance = 0.85
        elif alcohol_percentage >= 8:
            base_chance = 0.95

        age_factor = 1 - (self.age / self.max_age)
        stamina_factor = self.stamina / 100

        survival = base_chance * age_factor * stamina_factor
        return max(0, min(survival, 1))

    def can_reproduce(self, current_time, alcohol_percentage):
        if self.divisions >= self.max_divisions:
            return False
        if alcohol_percentage >= 10:
            return False
        return current_time % self.reproduction_interval == 0

    def reproduce(self):
```

```
self.divisions += 1
return YeastCell(
    max_age=self.max_age,
    max_divisions=self.max_divisions,
    reproduction_interval=self.reproduction_interval,
    alcohol_production=self.alcohol_production,
    lethal_alcohol=self.lethal_alcohol
)

def step(self, time_step):
    self.age += time_step
```

In de simulatie kan dan per tijdstap voor elke gistcel eerst **step()** uitgevoerd worden, daarna wordt **chance_of_survival()** gecontroleerd en tenslotte worden met **can_reproduce()** en **reproduce()** nieuwe cellen aangemaakt.

Visuele simulatie in Pygame

(suggestie van co-pilot)

“Pygame is een goede keuze als je de simulatie zichtbaar en intuïtief wilt maken. Je kunt daarmee tegelijk de populatie van gistcellen animeren en de belangrijkste waarden, zoals tijd en alcoholpercentage, live tonen op het scherm.

Voorstel voor de schermindeling

- *Linkerkant van het scherm: een vat of simulatiegebied met gistcellen als stippen.*
- *Rechterkant van het scherm: live statistieken zoals aantal levende cellen, alcoholpercentage en verstreken tijd.*
- *Kleurgebruik: lichte stippen voor gezonde cellen, oranje of rood voor gestreste cellen en eventueel grijs voor dode cellen.*

Wat elke gistcel minimaal nodig heeft

- *x- en y-positie op het scherm*
- *leeftijd*
- *aantal delingen*
- *stamina of conditie*
- *status levend of dood*

Wat er per frame gebeurt

- *achtergrond tekenen*
- *alle levende cellen updaten*
- *voortplanting laten gebeuren*
- *alcoholpercentage verhogen*
- *de gistcellen tekenen*
- *de statistieken op het scherm tonen*

Belangrijk is het onderscheid tussen simulatie- en schermtijd. De simulatie kan bijvoorbeeld eenmaal per paar frames een echte tijdstap uitvoeren, zodat de populatiegroei zichtbaar blijft en niet te snel uit de hand loopt.”

Eenvoudige Pygame-opzet

```
initialiseer pygame
maak venster

maak beginpopulatie van gistcellen
zet alcoholpercentage = 0
zet tijd = 0

while programma draait:
    verwerk input en events

    voer simulatiestap uit
    - update alle cellen
    - verwijder dode cellen
    - voeg nieuwe cellen toe
    - verhoog alcoholpercentage

    teken achtergrond
    teken alle gistcellen als cirkels
    teken tekst met tijd, populatiegrootte en alcoholpercentage

    update scherm
```

Voor een eerste werkende versie wordt begonnen met alleen bewegende gistcellen, eenvoudige groei, sterfte en een zichtbaar alcoholpercentage. Daarna kan het geheel eventueel uitgebreid worden met kleurverandering, grafieken en instelbare parameters.

Eerste werkende Pygame-code

Onderstaande code is een minimale werkende versie waarin gistcellen bewegen, ouder worden, zich kunnen delen en afsterven bij een hoger alcoholpercentage.

```
import pygame
import random

pygame.init()

WIDTH, HEIGHT = 1000, 600
SIM_WIDTH = 700
screen = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("Yeast Simulation")
clock = pygame.time.Clock()
font = pygame.font.SysFont(None, 28)

class YeastCell:
    def __init__(self, x, y):
        self.x = x
        self.y = y
        self.age = 0
```

```

        self.divisions = 0
        self.max_age = 120
        self.max_divisions = 25
        self.alive = True

    def step(self, alcohol):
        self.age += 1
        if self.age > self.max_age:
            self.alive = False
            return

        survival = 0.99
        if alcohol >= 12:
            survival = 0.2
        elif alcohol >= 10:
            survival = 0.85
        elif alcohol >= 8:
            survival = 0.95

        if random.random() > survival:
            self.alive = False

    def can_divide(self, alcohol):
        return self.alive and self.divisions < self.max_divisions and alcohol < 10
and random.random() < 0.03

    def divide(self):
        self.divisions += 1
        nx = self.x + random.randint(-12, 12)
        ny = self.y + random.randint(-12, 12)
        nx = max(10, min(SIM_WIDTH - 10, nx))
        ny = max(10, min(HEIGHT - 10, ny))
        return YeastCell(nx, ny)

    def draw(self, surface, alcohol):
        if alcohol < 8:
            color = (240, 240, 180)
        elif alcohol < 10:
            color = (255, 180, 80)
        else:
            color = (200, 80, 80)
        pygame.draw.circle(surface, color, (int(self.x), int(self.y)), 4)

cells = [YeastCell(random.randint(20, SIM_WIDTH - 20), random.randint(20, HEIGHT -
20)) for _ in range(80)]
alcohol = 0.0
time_step = 0
running = True

while running:
    clock.tick(60)

```

```

for event in pygame.event.get():
    if event.type == pygame.QUIT:
        running = False

if pygame.time.get_ticks() % 200 == 0:
    time_step += 1
    new_cells = []
    alive_cells = []

    for cell in cells:
        cell.step(alcohol)
        if cell.alive:
            alive_cells.append(cell)
            if cell.can_divide(alcohol):
                new_cells.append(cell.divide())

    cells = alive_cells + new_cells
    alcohol += len(alive_cells) * 0.0002
    if alcohol > 15:
        alcohol = 15

screen.fill((30, 18, 10))
pygame.draw.rect(screen, (60, 30, 15), (0, 0, SIM_WIDTH, HEIGHT))
pygame.draw.rect(screen, (20, 20, 30), (SIM_WIDTH, 0, WIDTH - SIM_WIDTH,
HEIGHT))

for cell in cells:
    cell.draw(screen, alcohol)

lines = [
    f"Tijd: {time_step}",
    f"Levende cellen: {len(cells)}",
    f"Alcohol: {alcohol:.2f}%"
]

for i, line in enumerate(lines):
    text = font.render(line, True, (240, 240, 240))
    screen.blit(text, (720, 40 + i * 40))

pygame.display.flip()

pygame.quit()

```

De instelling `clock.tick(60)` bleek erg optimistisch, daarmee werd de simulatie zo traag dat het op realtime begon te lijken. Fors opgehoogd in de code => acceptabele snelheid.

Klasse YeastCellSprite

Als je later een nettere scheiding wilt tussen simulatie en visualisatie, kun je de logica van de gistcel loshouden van de manier waarop die op het scherm wordt getekend.

Daarvoor is een spriteklasse handig.

```
import pygame

class YeastCellSprite(pygame.sprite.Sprite):
    def __init__(self, yeast_cell):
        super().__init__()
        self.yeast_cell = yeast_cell
        self.image = pygame.Surface((10, 10), pygame.SRCALPHA)
        self.rect = self.image.get_rect(center=(yeast_cell.x, yeast_cell.y))
        self.update_color(0)

    def update_color(self, alcohol):
        self.image.fill((0, 0, 0, 0))
        if alcohol < 8:
            color = (240, 240, 180)
        elif alcohol < 10:
            color = (255, 180, 80)
        else:
            color = (200, 80, 80)
        pygame.draw.circle(self.image, color, (5, 5), 4)

    def update(self, alcohol):
        self.rect.center = (self.yeast_cell.x, self.yeast_cell.y)
        self.update_color(alcohol)
```

Voorgestelde projectstructuur

Een eenvoudige bestandsindeling maakt het project later makkelijker uit te breiden en te onderhouden.

- **main.py**: start Pygame, maakt het venster, verwerkt events en roept de simulatie en tekenlogica aan.
- **yeast.py**: bevat de klasse **YeastCell** en eventueel later ook **YeastCellSprite**.
- **simulation.py**: beheert de populatie, tijdstappen, alcoholtoename en statistieken.

```
project_map/  
|-- main.py  
|-- yeast.py  
|-- simulation.py
```

Een logische volgende stap was om de eerste werkende code op te splitsen over deze drie bestanden, zodat de simulatiecode overzichtelijker wordt en later makkelijker grafieken, instellingen of meerdere scenario's kunnen worden toegevoegd.

Code opsplitsen over main.py, yeast.py en simulation.py

Hieronder staat een eenvoudige verdeling van verantwoordelijkheden, zodat logica, simulatie en weergave los van elkaar blijven.

yeast.py

```
import random

class YeastCell:
    def __init__(self, x, y, vx=None, vy=None):
        self.x = x
        self.y = y
        self.vx = vx if vx is not None else random.uniform(-0.8, 0.8)
        self.vy = vy if vy is not None else random.uniform(-0.8, 0.8)
        self.age = 0
        self.divisions = 0
        self.max_age = 120
        self.max_divisions = 25
        self.alive = True

    def move(self, width, height):
        self.x += self.vx
        self.y += self.vy
        if self.x < 10 or self.x > width - 10:
            self.vx *= -1
        if self.y < 10 or self.y > height - 10:
            self.vy *= -1

    def step(self, alcohol):
        self.age += 1
        if self.age > self.max_age:
            self.alive = False
            return
        survival = 0.99
        if alcohol >= 12:
            survival = 0.20
        elif alcohol >= 10:
            survival = 0.85
        elif alcohol >= 8:
            survival = 0.95
        if random.random() > survival:
            self.alive = False

    def can_divide(self, alcohol):
        return self.alive and self.divisions < self.max_divisions and alcohol < 10
        and random.random() < 0.03

    def divide(self, width, height):
        self.divisions += 1
```

```
nx = max(10, min(width - 10, self.x + random.randint(-12, 12)))
ny = max(10, min(height - 10, self.y + random.randint(-12, 12)))
return YeastCell(nx, ny)
```

simulation.py

```
from yeast import YeastCell
import random

class Simulation:
    def __init__(self, width, height, start_count=80):
        self.width = width
        self.height = height
        self.cells = [YeastCell(random.randint(20, width - 20), random.randint(20,
height - 20)) for _ in range(start_count)]
        self.alcohol = 0.0
        self.time_step = 0
        self.population_history = []
        self.alcohol_history = []

    def update(self):
        self.time_step += 1
        new_cells = []
        alive_cells = []

        for cell in self.cells:
            cell.move(self.width, self.height)
            cell.step(self.alcohol)
            if cell.alive:
                alive_cells.append(cell)
                if cell.can_divide(self.alcohol):
                    new_cells.append(cell.divide(self.width, self.height))

        self.cells = alive_cells + new_cells
        self.alcohol += len(alive_cells) * 0.0002
        self.alcohol = min(self.alcohol, 15)
        self.population_history.append(len(self.cells))
        self.alcohol_history.append(self.alcohol)
```

main.py

```
import pygame
from simulation import Simulation

pygame.init()

WIDTH, HEIGHT = 1000, 600
```

```

SIM_WIDTH = 700
screen = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("Yeast Simulation")
clock = pygame.time.Clock()
font = pygame.font.SysFont(None, 28)

sim = Simulation(SIM_WIDTH, HEIGHT)
running = True

while running:
    clock.tick(60)
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    if pygame.time.get_ticks() % 200 == 0:
        sim.update()

    screen.fill((30, 18, 10))
    pygame.draw.rect(screen, (60, 30, 15), (0, 0, SIM_WIDTH, HEIGHT))
    pygame.draw.rect(screen, (20, 20, 30), (SIM_WIDTH, 0, WIDTH - SIM_WIDTH,
HEIGHT))

    for cell in sim.cells:
        if sim.alcohol < 8:
            color = (240, 240, 180)
        elif sim.alcohol < 10:
            color = (255, 180, 80)
        else:
            color = (200, 80, 80)
        pygame.draw.circle(screen, color, (int(cell.x), int(cell.y)), 4)

    lines = [
        f"Tijd: {sim.time_step}",
        f"Levende cellen: {len(sim.cells)}",
        f"Alcohol: {sim.alcohol:.2f}%"
    ]
    for i, line in enumerate(lines):
        text = font.render(line, True, (240, 240, 240))
        screen.blit(text, (720, 40 + i * 40))

    pygame.display.flip()

pygame.quit()

```

Pygame-code verbeteren met echte beweging

(suggesties van co-pilot)

“Om de simulatie levendiger te maken, kun je gistcellen een eigen snelheid geven en die bij botsen tegen de rand laten omkeren. Ook kun je de snelheid iets verlagen als het alcoholpercentage stijgt, zodat stress direct zichtbaar wordt.

```
def move(self, width, height, alcohol):
    stress_factor = max(0.3, 1 - alcohol / 15)
    self.x += self.vx * stress_factor
    self.y += self.vy * stress_factor

    if self.x < 10 or self.x > width - 10:
        self.vx *= -1
    if self.y < 10 or self.y > height - 10:
        self.vy *= -1
```

In simulation.py vervang je dan cell.move(self.width, self.height) door cell.move(self.width, self.height, self.alcohol).”

Klein grafiekvenster in Pygame

(suggesties van co-pilot)

“Omdat simulation.py al een geschiedenis van populatie en alcohol opslaat, kun je rechts in het scherm een eenvoudige lijnengrafiek tekenen.

```
def draw_graph(screen, values, color, x, y, width, height, max_value):
    if len(values) < 2 or max_value == 0:
        return
    step_x = width / max(1, len(values) - 1)
    points = []
    for i, value in enumerate(values[-100:]):
        px = x + i * step_x
        py = y + height - (value / max_value) * height
        points.append((px, py))
    if len(points) > 1:
        pygame.draw.lines(screen, color, False, points, 2)
```

Met deze uitbreiding zie je niet alleen de cellen bewegen, maar ook meteen of de populatie een groeifase, piek en afname laat zien, en hoe snel het alcoholpercentage oploopt.”